

A Probabilistic and Explainable Tool for Context-Aware Multimodal Trip Planning Using Generative AI

Internship Report

Submitted in partial fulfilment of the requirements of the
GRISHMA Summer Research Programme 2026

Submitted by

Nikhil Kumar

GRISHMA-STU-2026-18054

Under the supervision of

Dr. Nandan Maiti

Assistant Professor

Department of Civil Engineering, IIT Kharagpur



Department of Civil Engineering
Indian Institute of Technology Kharagpur

May – July 2026

DECLARATION

I, **Nikhil Kumar (GRISHMA-STU-2026-18054)**, declare that the internship report titled “**A Probabilistic and Explainable Tool for Context-Aware Multimodal Trip Planning Using Generative AI**” submitted to **Indian Institute of Technology Kharagpur** as part of the **GRISHMA Summer Research Programme 2026** is based on original work carried out by me under the supervision of **Dr. Nandan Maiti**, Department of Civil Engineering, IIT Kharagpur.

This report has not been submitted elsewhere for any degree, diploma, or qualification. All external sources used have been duly cited.

Kharagpur – 721302
July 2026

Nikhil Kumar
GRISHMA-STU-2026-18054

ABSTRACT

Most navigation systems today update routes only after a disruption has already been detected by sensors – they are reactive by design. This project builds an **anticipatory** alternative for Kolkata, India.

The system, **traffic-ai**, pulls unstructured text from nine live sources (news RSS feeds, TomTom traffic incidents, official advisories, weather APIs) and uses a Large Language Model to extract structured disruption events from that text *before* a commuter travels. Each event carries a severity score σ and a confidence weight κ . A Temporal Heterogeneous Graph Neural Network (HGNN) then adjusts those confidence values using city-wide spatial context. A Bayesian fusion layer converts the signals into per-road disruption probabilities, which are used to colour-code and rank candidate routes on a Leaflet.js map.

The system supports eight transport modes spanning drive, walk, bike, all five Kolkata Metro lines, and bus. The API is split into two steps: routes appear on screen within ~ 2 s while the slower LLM pipeline (~ 30 – 60 s) fills in risk scores in the background.

Aspect	Summary
Pilot city	Kolkata, India
Data sources	9 (structured + unstructured)
Transport modes	8
LLM backend	OpenRouter / Gemini / Ollama (configurable)
Graph model	Temporal HAN – 3 output heads
Probabilistic layer	Bayesian fusion with time-of-day priors

Contents

Title Page	i
Declaration	ii
Abstract	iii
1 Introduction	1
1.1 Context	1
1.2 Problem Statement	1
1.3 Research Hypotheses	1
1.4 Scope of Internship	2
1.5 Report Structure	2
2 System Architecture	3
2.1 Layer Overview	3
2.2 Data Ingestion Sources	3
2.3 Two-Step API Design	4
2.4 Transport Modes	4
2.5 API Endpoints	5
2.6 Project File Structure	5
3 Implementation	6
3.1 Layer 1 – LLM Disruption Intelligence	6
3.1.1 Pre-filter	6
3.1.2 LLM Chain	6
3.1.3 Extraction Schema	7
3.1.4 Risk Scoring	7
3.1.5 Temporal HGNN	8
3.2 Layer 2 – Bayesian Probabilistic Fusion	9
3.2.1 Bayesian Update	9
3.2.2 Time-of-Day Priors	9
3.2.3 Likelihood Ratios	9
3.2.4 HGNN Blend	9
3.3 Routing	10
3.3.1 Road Network (OSMnx [1])	10
3.3.2 Kolkata Metro	10
3.3.3 Bus Transit	10

3.3.4	Weather Severity Index	11
3.4	Database	11
4	Results and System Demonstration	12
4.1	Performance Metrics	12
4.2	UI Screenshots	12
4.3	Research Poster	14
5	Conclusion and Future Work	16
5.1	Summary of Contributions	16
5.2	Research Hypotheses – Status	16
5.3	Skills Acquired	17
5.4	Future Work	17

List of Figures

4.1	Step 1 — routes rendered before disruption analysis.	12
4.2	Step 2 — routes coloured by risk; disruption panel populated.	13
4.3	Metro overlay.	13
4.4	Bus transit overlay and routing result.	14
4.5	HGNN attention-based route explanation panel.	14

List of Tables

1.1	Reactive vs. anticipatory trip planning	1
1.2	Guiding hypotheses	1
1.3	Work carried out during the internship	2
2.1	System layer summary	3
2.3	Two-step request flow	4
2.4	Supported transport modes	4
2.5	Complete API surface	5
2.6	Key modules and their roles	5
3.1	Pre-LLM filter criteria	6
3.2	<code>TrafficEventSchema</code> fields	7
3.3	Multiplier definitions	7
3.4	Risk level thresholds	7
3.5	HGNN graph structure	8
3.6	HGNN output heads	8
3.7	Kolkata empirical baseline disruption probabilities by period	9
3.8	Severity likelihood ratios $p(S \mid Z_e=1)/p(S \mid Z_e=0)$	9
3.9	OSM graph coverage	10
3.10	Metro lines integrated	10
3.11	Bus transit module components	10
3.12	<code>traffic_events</code> table key column groups	11
4.1	System performance summary	12
5.1	Contributions made during the internship	16
5.2	Hypothesis status at end of internship	16
5.3	Technical skills developed	17
5.4	Planned extensions	17

Chapter 1

Introduction

1.1 Context

Kolkata’s road network carries millions of trips daily across a mix of private vehicles, metros, busses, and pedestrians. Disruptions – waterlogging, VIP convoys, accidents, road closures – are frequent and hard to predict from sensor data alone because a large share of disruption information first appears in news articles and official advisories, not on roads.

Existing navigation tools (Google Maps, Apple Maps) react to congestion after it shows up in GPS probe data, often minutes to hours after the event begins. This project asks whether reading that unstructured text *before travel* can produce better route recommendations.

1.2 Problem Statement

Dimension	Current systems	This project
Data types used	Structured only	Structured + unstructured
Timing	After disruption	Before disruption
Objective	Minimise travel time	Minimise risk-adjusted time
Uncertainty	Deterministic	Probabilistic
Explainability	None	HGNN attention weights
Multimodal	Limited	8 modes inc. metro and bus

Table 1.1: Reactive vs. anticipatory trip planning

1.3 Research Hypotheses

ID	Hypothesis
H1	LLM-derived contextual signals improve disruption detection accuracy over structured-data-only baselines.
H2	Probabilistic, risk-aware routing improves travel-time reliability under uncertain conditions.
H3	Exposing HGNN attention weights as route explanations increases commuter trust in system recommendations.

Table 1.2: Guiding hypotheses

1.4 Scope of Internship

Table 1.3: Work carried out during the internship

#	Contribution	Status
1	LLM disruption extraction pipeline (Layer 1)	Done
2	Multi-factor confidence scoring + impact duration	Done
3	Temporal HGNN – training loop + inference	Done
4	Bayesian probabilistic fusion (Layer 2)	Done
5	Weather Severity Index per route	Done
6	Kolkata Metro integration (5 lines, timetable, next-train)	Done
7	Bus transit graph + BFS routing engine	Done
8	Multimodal routing orchestration	Done
9	FastAPI backend (11 endpoints)	Done
10	Leaflet.js frontend + map overlays	Done
11	CVaR risk-aware routing (Layer 3)	In progress

1.5 Report Structure

Chapter	Content
2	System architecture, data sources, API design
3	Implementation – Layers 1, 2, routing, database
4	Results, UI screenshots, performance metrics
5	Conclusion and future work

Chapter 2

System Architecture

2.1 Layer Overview

The system is organised into three processing layers. Layers 1 and 2 are complete; Layer 3 is in progress.

Layer	Function	Key output	Status
1	LLM extraction + HGNN refinement	σ, κ per event	Complete
2	Bayesian fusion	π_e^{post} per road edge	Complete
3	CVaR risk-aware routing + NL explainability	optimal route + justification	In progress

Table 2.1: System layer summary

End-to-end data flow:

Sources $\xrightarrow{\text{ingest}}$ Pre-filter $\xrightarrow{\text{LLM}}$ Events DB $\xrightarrow{\text{HGNN}}$ Conf. $\xrightarrow{\text{adjust}}$ Bayes $\xrightarrow{\text{Road probs}}$ Road probs
 $\xrightarrow{\text{score}}$ Ranked routes $\rightarrow$ Leaflet map

2.2 Data Ingestion Sources

Source	Type	What it provides
TomTom Incidents API v5	Unstructured, live	Accidents, closures, works with GPS coords; 2,500 free req/day
Google News RSS when:2d	Unstructured	City-wide Kolkata disruption news, last 2 days
NewsAPI	Unstructured	Per-road keyword search, last 24 h
OpenWeatherMap	Structured, live	Precipitation, wind, visibility for WSI
Kolkata Police (via RSS)	Unstructured	Official traffic advisories
KMRC (metro authority)	Unstructured	Metro service disruption notices
WB Disaster Management	Unstructured	Flood and cyclone alerts

Source		Type	What it provides
Indian (erail.in)	Railways	Structured	Delayed trains at Howrah / Seal-dah
KMC (RSS)	Waterlogging	Unstructured	Waterlogging hotspot reports

2.3 Two-Step API Design

The most important UX decision in the system is decoupling fast route rendering from slow LLM analysis.

Step 1 — POST /api/route (~2 s)	Step 2 — POST /api/disruptions (~30–60 s)
OSMnx computes 2–3 alternative routes on the cached Kolkata graph. GeoJSON returned immediately. All routes shown green (no risk data yet).	Fetches all 9 sources. LLM extracts events. HGNN adjusts confidence. Bayesian fusion runs. Routes re-coloured by risk; markers placed on map.

Table 2.3: Two-step request flow

2.4 Transport Modes

Mode key	Description	Graph used
drive	Car routing	OSMnx drive
walk	Pedestrian routing	OSMnx walk
bike	Cycling routing	OSMnx bike
metro	Metro-only (5 lines, BFS)	Metro graph
metro+walk	Metro + walk access/egress legs	Both
metro+bike	Metro + cycling access/egress legs	Both
metro+drive	Metro + drive access/egress legs	Both
bus	Kolkata bus network (BFS)	Bus graph

Table 2.4: Supported transport modes

2.5 API Endpoints

Method	Endpoint	Description
GET	/	Leaflet.js frontend
GET	/api/locations	Localities, metro stations, bus stops
POST	/api/route	Step 1 — fast route computation
POST	/api/disruptions	Step 2 — LLM + HGNN + Bayes scoring
POST	/api/explain-route	HGNN attention-based explanation
GET	/api/metro-overlay	GeoJSON of all 5 metro lines
GET	/api/metro-lines	Timetable summary per line
GET	/api/next-metro/{station}	Next n trains (live IST)
GET	/api/bus-overlay	GeoJSON of all bus stops
GET	/api/bus-network	Full bus route + stop data
GET	/api/hgnn-status	Model readiness check

Table 2.5: Complete API surface

2.6 Project File Structure

File / module	Role
app/api.py	FastAPI server, all 11 endpoints
app/config.py	Reads <code>.env</code> ; all feature flags
app/ingestion/	One file per data source
app/llm/extractor.py	LangChain LCEL chain + JSON repair
app/llm/schema.py	TrafficEventSchema (Pydantic)
app/hgnn/model.py	Temporal HAN, 3 output heads
app/hgnn/integration.py	Drop-in helpers: enhance, score, cascade
app/hgnn/trainer.py	Training loop (MSE + CrossEntropy)
app/fusion/bayesian_fusion.py	Bayesian update + HGNN blend
app/routing/route_engine.py	OSMnx k -shortest paths, cache
app/routing/metro_timetable.py	5-line metro data + next-train
app/transit/	Bus graph, BFS engine, GeoJSON overlay
app/weather/	Route-point sampling + WSI
app/scoring/	$\sigma \times \kappa$ formula, confidence
app/database/models.py	SQLAlchemy schema + auto-migration
app/static/	Leaflet.js [2] frontend (HTML/JS/CSS)

Table 2.6: Key modules and their roles

Chapter 3

Implementation

3.1 Layer 1 – LLM Disruption Intelligence

3.1.1 Pre-filter

Every article is checked against three criteria before reaching the LLM. Articles that fail any check are dropped without spending an API call.

Check	Threshold	Action on fail
Age	> 2 days old	Discard
Geography	Outside Kolkata bbox	Discard
Relevance	No transport keywords	Discard

Table 3.1: Pre-LLM filter criteria

3.1.2 LLM Chain

Articles that pass the pre-filter are sent to a LangChain LCEL chain [3] configured with `with_structured_output(TrafficEventSchema)`. The chain returns a typed Pydantic object directly. Three fallback stages handle malformed output:

1. Strip markdown fences, extract JSON object.
2. Fix missing colons: `{"key", "val"} → {"key": "val"}`.
3. Regex field extraction as last resort.

If the LLM returns no location and $\kappa \geq 0.6$, a second focused location-recovery prompt is sent. If $\kappa < 0.6$ the event is discarded.

3.1.3 Extraction Schema

Field	Type	Notes
event_type	Enum (13 vals)	accident, congestion, road_closure, construction, protest, weather, waterlogging, vip_movement, metro_disruption, train_delay, transport_strike, diversion, unknown
severity σ	low/med/high	Mapped to 2 / 5 / 10
confidence κ	float [0,1]	Adjusted by source reliability + age
location	str	Mandatory; event dropped if absent
road_name	str or null	Specific road if extractable
is_future_event	bool	Planned vs. currently active
impact_duration_mins	int	Estimated disruption length
location_source	Enum	direct / road_name / landmark / coordinates / llm_inferred

Table 3.2: TrafficEventSchema fields

3.1.4 Risk Scoring

$$\text{weighted_score} = \sigma \times \kappa \times d_m \times r_m \quad (3.1)$$

Symbol	Name	Range	Rule
d_m	Duration multiplier	1.0–2.0	≥ 1 day $\rightarrow 2.0$; ≥ 8 h $\rightarrow 1.5$; ≥ 2 h $\rightarrow 1.2$
r_m	Recency multiplier	0.5–1.0	≤ 1 h $\rightarrow 1.0$; ≤ 6 h $\rightarrow 0.9$; ≤ 24 h $\rightarrow 0.75$; older $\rightarrow 0.5$

Table 3.3: Multiplier definitions

$$\text{route_risk} = \sum_{\text{matched active events}} \text{weighted_score} \quad (3.2)$$

Level	Threshold	Map colour	Hex
CRITICAL	≥ 25	Deep purple	#8e44ad
HIGH	≥ 12	Red	#e74c3c
MODERATE	≥ 5	Orange	#f39c12
LOW	> 0	Yellow-green	#27ae60
CLEAR	$= 0$	Green	#27ae60

Table 3.4: Risk level thresholds

$$\text{best route} = \arg \min \left(t_{\text{travel}} \times \left(1 + \frac{\text{route_risk}}{10} \right) \right) \quad (3.3)$$

3.1.5 Temporal HGNN

The Heterogeneous Graph Neural Network refines κ values using city-wide graph context after the LLM extraction step [4].

Node type	Feature dim	Key features
Event	24	severity, κ , event type, age, temporal encoding
Road	6	historical disruption rate, rush-hour ratio, propagation score
Source	–	reliability score per ingestion source
Location	–	spatial cluster membership

Table 3.5: HGNN graph structure

Head	Output	Shape	Loss
1	Road disruption probability $\hat{\pi}_e$	$(N_{\text{road}},)$	MSE
2	Event confidence adjustment $\hat{\kappa}_i$	$(N_{\text{event}},)$	MSE
3	Event severity prediction	$(N_{\text{event}}, 3)$	CrossEntropy

Table 3.6: HGNN output heads

$$\kappa_{\text{final}} = 0.75 \times \kappa_{\text{rule-based}} + 0.25 \times \hat{\kappa}_{\text{HGNN}} \quad (3.4)$$

Severity override from Head 3 requires HGNN confidence ≥ 0.85 – a threshold set conservatively because the HGNN only sees tabular features while the LLM has read the full article text.

Cascade detection. After scoring, the HGNN identifies roads *not* on the requested route but spatially adjacent to affected roads (probability > 0.45). Up to 5 extra roads are added to the event matching pool, capturing knock-on effects (e.g. Howrah Bridge blocked $\Rightarrow$ Strand Road likely congested).

Training the model:

Listing 3.1: HGNN training command

```
cd app
python -m hgnn.trainer                                # default 100 epochs
python -m hgnn.trainer --epochs 120 --lr 5e-4
```

3.2 Layer 2 – Bayesian Probabilistic Fusion

3.2.1 Bayesian Update

$$\pi_e^{\text{post}}(t) = \frac{p(S \mid Z_e = 1) \cdot \pi_e^{\text{prior}}(t)}{\sum_{z \in \{0,1\}} p(S \mid Z_e = z) \cdot \pi_e^{\text{prior}}(t)} \quad (3.5)$$

where $Z_e(t)$ is the binary disruption state of edge e , $\pi_e^{\text{prior}}(t)$ is the time-of-day baseline probability, and S is the Layer 1 signal (σ, κ) .

3.2.2 Time-of-Day Priors

Period	π_e^{prior}
Midnight – 5 am (off-peak)	0.01 – 0.03
Morning rush (7 – 10 am)	0.12 – 0.20
Midday (10 am – 4 pm)	0.12 – 0.15
Evening rush (5 – 7 pm)	0.18 – 0.22
Late evening (8 – 11 pm)	0.06 – 0.14

Table 3.7: Kolkata empirical baseline disruption probabilities by period

3.2.3 Likelihood Ratios

Severity	σ	Ratio
High	10	8.0
Medium	5	4.0
Low	2	2.0

Table 3.8: Severity likelihood ratios $p(S \mid Z_e=1)/p(S \mid Z_e=0)$

Each ratio is further scaled by κ before the update – a high-confidence signal shifts the posterior more than a low-confidence one.

3.2.4 HGNN Blend

$$\pi_e^{\text{final}} = 0.70 \times \pi_e^{\text{post}} + 0.30 \times \hat{\pi}_e^{\text{HGNN}} \quad (3.6)$$

Roads with no matched events receive the time-of-day prior only.

3.3 Routing

3.3.1 Road Network (OSMnx [1])

Parameter	Value
Network type	Drive / walk / bike (separate graphs)
Cache format	Pickle (.pkl) – loads in ~ 2 s
First-run download	~ 30 – 60 s (one-time)
Alternative routes	Yen’s k -shortest paths
Min. route divergence	20% (configurable)
Kolkata localities	15 pre-defined locations in dropdown

Table 3.9: OSM graph coverage

3.3.2 Kolkata Metro

Line	Corridor	Colour	Status
Blue (Line 1)	Dakshineswar – Kavi Subhas	Blue	Operational
Green (Line 2)	Howrah Maidan – Sector V	Green	Operational
Purple (Line 3)	Joka – Taratala (Phase 1)	Purple	Operational
Orange (Line 4)	New Garia – Airport	Orange	Partial
Yellow (Line 5)	Noapara – Barasat	Yellow	Partial

Table 3.10: Metro lines integrated

Routing uses BFS with interchange handling. The `next_trains_at_station()` function computes live departures from the current IST time.

3.3.3 Bus Transit

File	Role
<code>transit/data/bus_stops.json</code>	Stop ID, name, lat, lon for all major Kolkata stops
<code>transit/data/bus_routes.json</code>	Route number, type (AC / Government / Non-AC / Mini), ordered stop sequence
<code>transit/bus_graph.py</code>	Directed graph: nodes = stops, edges = consecutive stops
<code>transit/bus_engine.py</code>	BFS route finder, minimises number of transfers
<code>transit/bus_overlay.py</code>	GeoJSON FeatureCollection for Leaflet map layer

Table 3.11: Bus transit module components

3.3.4 Weather Severity Index

OpenWeatherMap [5] is queried at multiple sampled points along each route. The WSI aggregates precipitation, wind speed, visibility, and condition codes into a single scalar $\in [0, 10]$, reported alongside the disruption risk score for each route.

3.4 Database

Group	Columns
Source metadata	source, source_url, tomtom_url, raw_text, fetched_at, published_date
LLM outputs	event_type, location, road_name, severity, confidence, reason, is_future_event
Location inference	lat, lon, location_inferred, location_source
HGNN feedback	hgnn_confidence, hgnn_severity, hgnn_multiplier, severity_corrected
Impact duration	impact_duration_mins, impact_duration_label, estimated_end_time, duration_source

Table 3.12: `traffic_events` table key column groups

Auto-migration. On every startup, `PRAGMA table_info` is checked and any missing columns are added via `ALTER TABLE` without dropping existing data. This allows teammates to pull new code without losing their local database.

Portability. Switching from SQLite to PostgreSQL requires only changing `DATABASE_URL` in `.env`.

Chapter 4

Results and System Demonstration

4.1 Performance Metrics

Metric	Observed value
Route computation (/api/route)	~ 2 s
LLM pipeline (/api/disruptions)	30 – 60 s
OSM graph load from pickle cache	~ 2 s
OSM graph first-time download	~ 30 – 60 s (once)
Data sources integrated	9
Transport modes supported	8
Pre-defined Kolkata localities	15 + metro stations + bus stops
Metro lines	5
Event types extractable	13
HGNN output heads	3
TomTom free-tier capacity	2,500 req/day
LLM rate-limit handling	Exponential backoff on 429

Table 4.1: System performance summary

4.2 UI Screenshots

Instructions for compilation: place your .png screenshots in the same folder as the .tex files and uncomment the relevant \includegraphics line inside each figure environment.

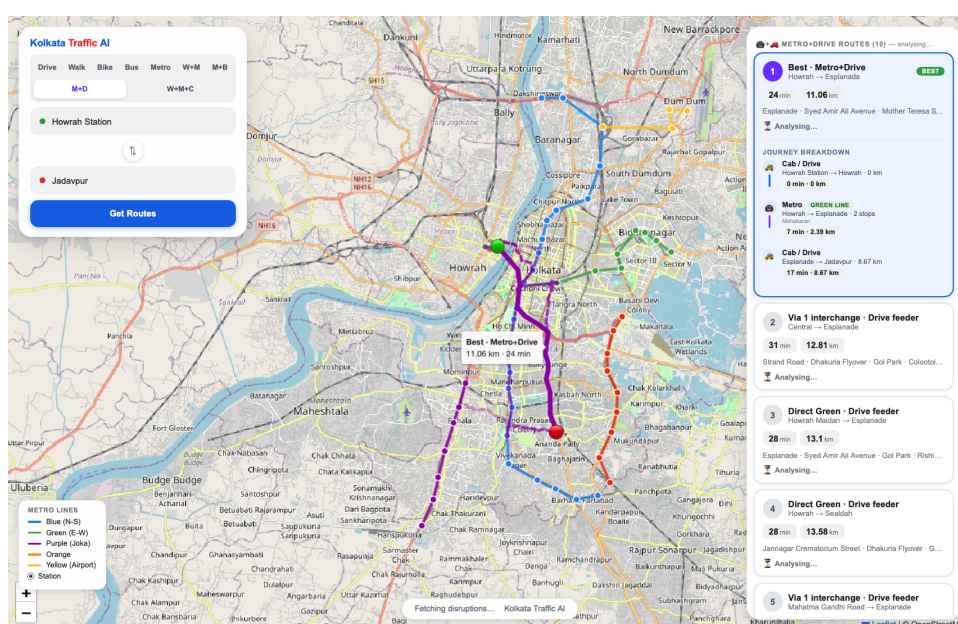


Figure 4.1: Step 1 — routes rendered before disruption analysis.

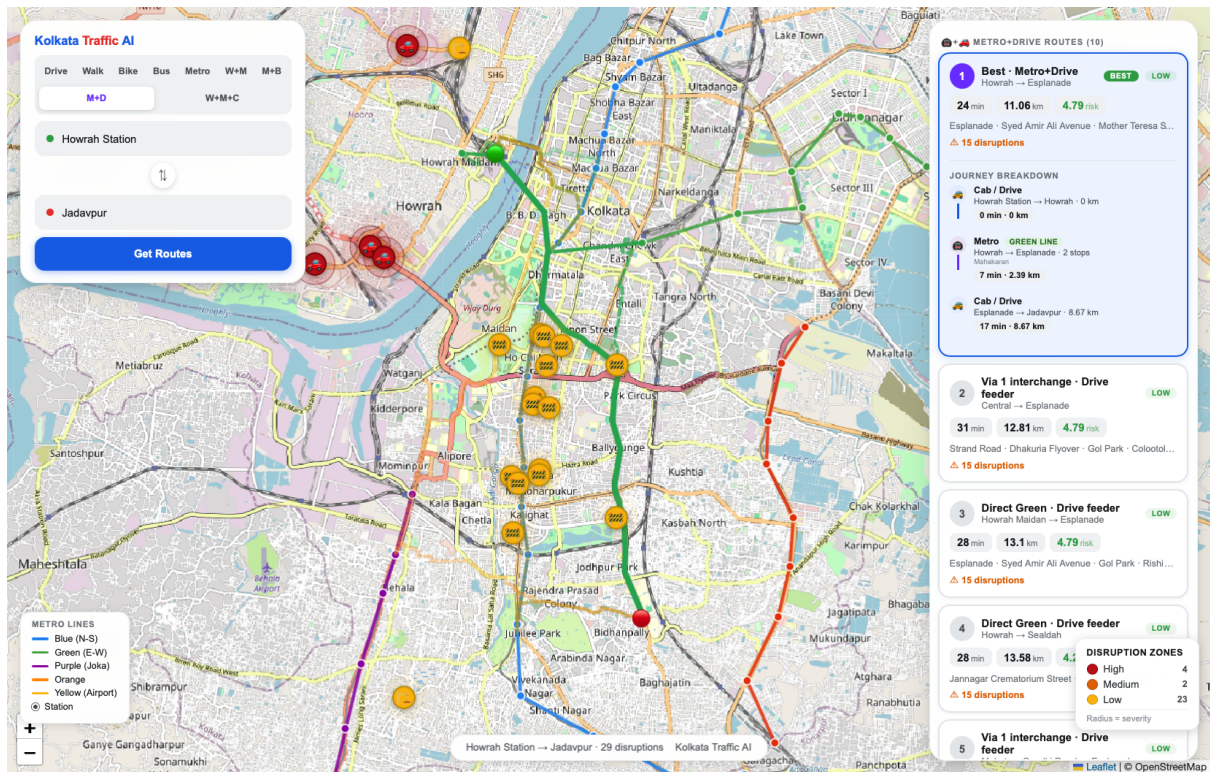


Figure 4.2: Step 2 — routes coloured by risk; disruption panel populated.



Figure 4.3: Metro overlay.

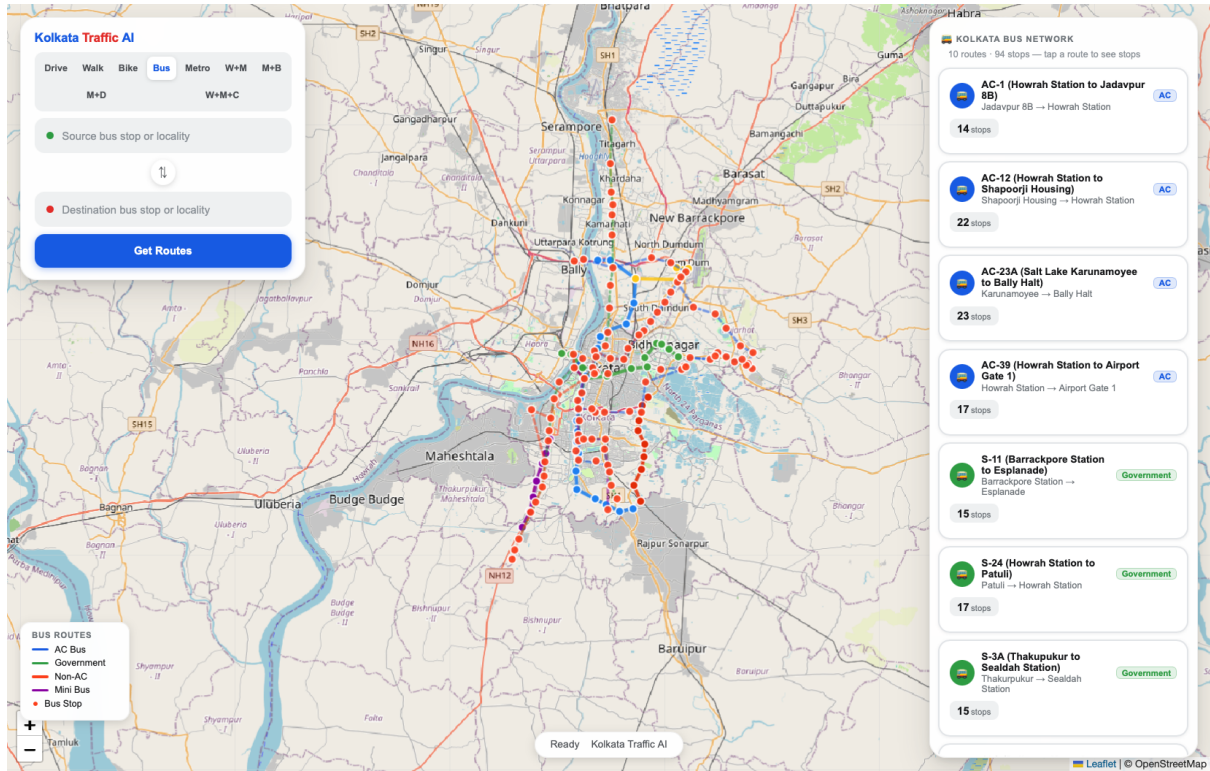


Figure 4.4: Bus transit overlay and routing result.

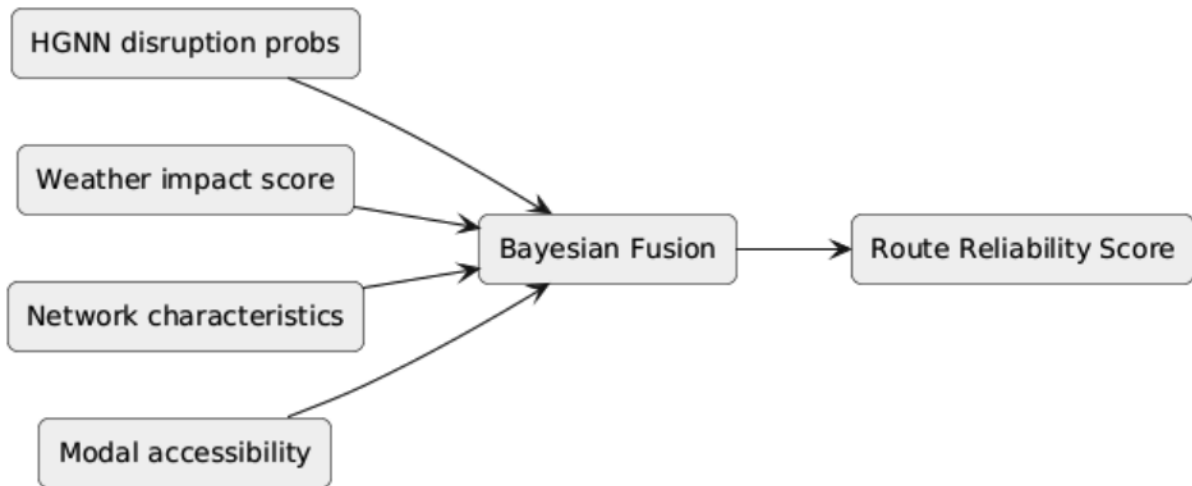


Figure 4.5: HGNN attention-based route explanation panel.

4.3 Research Poster

The poster below was submitted as part of the GRISHMA programme deliverables. It summarises the system architecture, methodology, and key findings.

UrbanMind: A Unified Explainable GenAI Framework for Anticipatory, Risk-Aware Urban Multimodal Mobility Management

Nikhil Kumar¹, Ruchi Mishra², Lakshya Sharma³, Aayush Mathur³,
Dr. Nandan Maiti^{4*}



¹IIT Kottayam, ²Banasthali Vidyapeeth
³Rajasthan Technical University
⁴Indian Institute of Technology Kharagpur
*nandan@civil.iitkgp.ac.in

PRAVAH-2026-1928

Introduction

Urban mobility in Indian cities is highly dynamic, disrupted by weather, public events, infrastructure works, and socioeconomic activities. While digital mapping and traffic tools are widespread, most remain reactive - relying on structured traffic records and transit schedules. They often miss contextual signals from traffic APIs, social media, public notices, and weather alerts, causing less reliable routing decisions in rapidly changing urban environments.

Moving from reactive navigation to proactive disruption-aware mobility.

Problem Statement

Pain points:

Reactive Routing

Current systems respond after disruptions occur, rather than predicting mobility risks in advance.

Unused Real-Time Signals

Valuable information from traffic APIs, news, weather alerts, and social sources remains under-utilised.

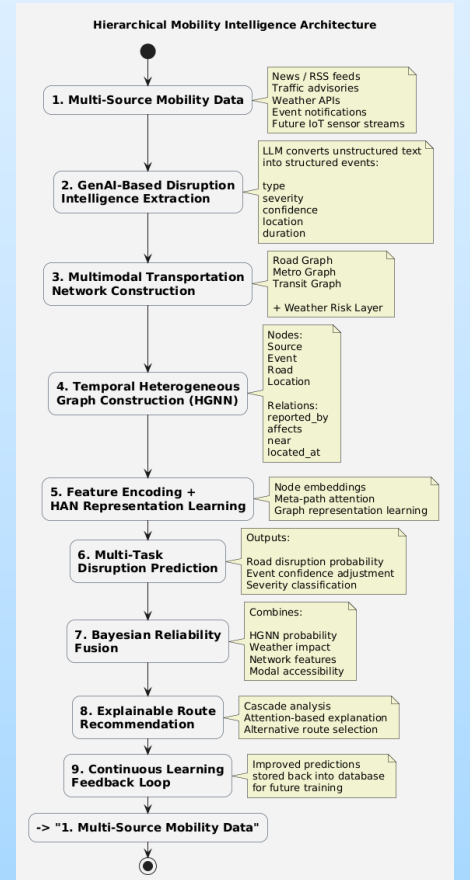
Limited Explainability

Route decisions often lack transparency, reducing user trust in intelligent mobility systems.

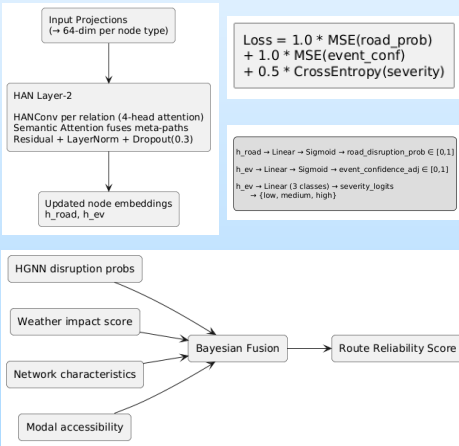
How can real-time urban data be converted into explainable intelligence for proactive and personalized mobility decisions?

System Architecture

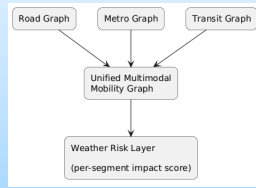
UrbanMind integrates heterogeneous mobility data with GenAI extraction, HGNN-based spatial reasoning, Bayesian risk fusion, and explainable route optimization to enable proactive multimodal travel decisions.



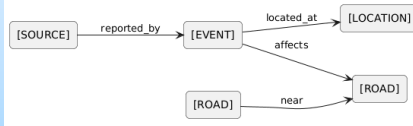
Heterogeneous Attention Network & Bayesian Reliability Fusion



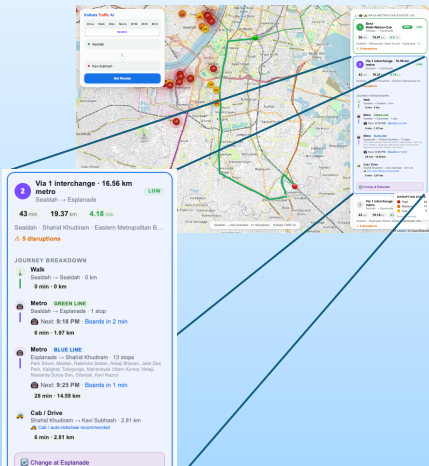
Multimodal Routing Engine



Temporal Heterogeneous Graph Construction (HGNN)



Prototype Demonstration



Positioning Against Existing Mapping Services

Aspect	Existing Systems	UrbanMind
Data Sources	Structured + Crowd	Structured + Unstructured
Event Handling	Reactive Updates	Future-aware Prediction
Uncertainty	Fixed Decisions	Bayesian Risk Score
Route Selection	Shortest Time	Risk + Reliability
Explainability	Limited	HGNN Attention
Mobility Modes	Limited	Multimodal Routing

Acknowledgement & References

We sincerely thank **Dr. Nandan Maiti**, Department of Civil Engineering, IIT Kharagpur, for his valuable guidance and mentorship throughout this work. We also acknowledge the support and collaboration of our teammates, and the innovative research environment provided through the **KRITI** initiative by IIT Kharagpur, which encouraged interdisciplinary problem solving and development of this project.

References:

1. Ganin et al., *Science Advances*, 3 (2017), 1701079
2. Maiti et al., *European Transport Research Review*, 17 (2025), 51
3. Chilukuri & Maiti, Indian Patent IN411487, 2022
4. Maiti et al., *Trans. Res. Part C*, 178 (2025), 105213
5. Wang et al., *IEEE Trans. ITS*, 14(3) (2013): 1331–1345
6. Qin et al., *Journal of Public Transportation*, 27 (2025): 100117

Chapter 5

Conclusion and Future Work

5.1 Summary of Contributions

Contribution	Outcome
LLM extraction pipeline	Structured disruption events from 9 live sources; 13 event types; mandatory location enforcement
Temporal HGNN	City-wide confidence refinement; 3-output-head model; cascade road detection; attention-based explainability
Bayesian probabilistic fusion	Per-road posterior disruption probabilities; time-of-day calibrated priors; HGNN blend
Multimodal routing	8 transport modes; full 5-line Kolkata Metro; BFS bus routing; OSMnx road-/walk/bike graphs
Weather risk assessment	Route-point sampling; Weather Severity Index (WSI) per candidate route
FastAPI backend	11 production endpoints; two-step design; 2 s initial render regardless of LLM latency
Frontend	Leaflet.js map; risk-coloured route lines; event markers; metro + bus overlays
Database	Auto-migrating SQLite schema; HGNN feedback loop;

Table 5.1: Contributions made during the internship

5.2 Research Hypotheses – Status

ID	Hypothesis	Status
H1	LLM signals improve disruption detection accuracy	Demonstrated qualitatively; ground-truth eval pending
H2	Risk-aware routing improves travel-time reliability	Layer 2 complete; Layer 3 (CVaR) in progress
H3	Explainable outputs enhance user trust	HGNN attention exposed via API; user study pending

Table 5.2: Hypothesis status at end of internship

5.3 Skills Acquired

Area	Specifics
LLM application dev	LangChain LCEL, prompt engineering, JSON repair, structured output
Graph neural networks	Heterogeneous graph construction, HAN layers, multi-task training (MSE + CrossEntropy)
Probabilistic ML	Bayesian inference, likelihood ratios, time-varying priors
Geospatial engineering	OSMnx, OpenStreetMap, GeoJSON, Leaflet.js, Nominatim geocoding
Backend development	FastAPI, Pydantic, SQLAlchemy, REST API design
Data engineering	Multi-source ingestion, RSS/web scraping, API rate-limit handling

Table 5.3: Technical skills developed

5.4 Future Work

#	Task	Details
1	Layer 3 — CVaR routing	Generalised edge cost combining travel time, risk, CO ₂ , and transfer penalty; CVaR optimisation for route selection
2	HGNN ground-truth training	Collect verified disruption labels from Kolkata Traffic Police; retrain severity and confidence heads on real data
3	GTFS-RT integration	Replace static metro/bus timetables with live GTFS-Realtime feeds
4	Natural language explanations	Layer 3 generates a plain-English justification per route recommendation (H3 full validation)
5	User trust study	Formal user experiment to validate H3 with Kolkata commuters
6	Multi-city deployment	Replace Kolkata OSM graph and transport datasets with those of another Indian metro for portability test

Table 5.4: Planned extensions

Bibliography

- [1] Geoff Boeing. OSMnx: New methods for acquiring, constructing, analysing, and visualising complex street networks. *Computers, Environment and Urban Systems*, 65:126–139, 2017.
- [2] Vladimir Agafonkin. Leaflet — mobile-friendly interactive maps. <https://leafletjs.com/>, 2024.
- [3] LangChain, Inc. LangChain documentation. <https://python.langchain.com/>, 2024.
- [4] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S. Yu. Heterogeneous graph attention network. In *The World Wide Web Conference (WWW)*, pages 2022–2032, 2019.
- [5] OpenWeather Ltd. OpenWeatherMap current weather API. <https://openweathermap.org/api>, 2024.
- [6] Nikhil Kumar. traffic-ai: Contextual probabilistic ai-enabled multimodal trip planning. <https://github.com/sawarn-nik/traffic-ai>, 2026.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [8] Daniel Work, Sebastien Blandin, Olli-Pekka Tossavainen, Benedetto Piccoli, and Alexandre Bayen. A traffic model for velocity data assimilation. *Applied Mathematics Research eXpress*, 1:1–35, 2010.
- [9] R. Tyrrell Rockafellar and Stanislav Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.
- [10] Sebastián Ramírez. FastAPI documentation. <https://fastapi.tiangolo.com/>, 2024.
- [11] TomTom International BV. TomTom traffic incidents API v5. <https://developer.tomtom.com>, 2024.
- [12] KMRC. Kolkata metro rail corporation — official website. <https://www.kmrc.in/>, 2024.
- [13] OpenStreetMap Foundation. Nominatim — OpenStreetMap geocoding. <https://nominatim.org/>, 2024.

- [14] SQLAlchemy Authors. SQLAlchemy — the python SQL toolkit. <https://www.sqlalchemy.org/>, 2024.